

Study Companion — Technical Code & Technology Reference Manual

Deep-Dive Guide: Technologies, Frameworks, Code Implementation & Architecture Interconnections

1. Technology Stack Breakdown & System Roles

Study Companion is built as a modern, decoupled 3-tier web application. Every technology component has been specifically chosen to ensure asynchronous non-blocking performance, strong type safety, strict subject isolation, and seamless real-time user interaction.

1. React 18 (Frontend Framework)

What it is: A declarative, component-based JavaScript UI library for building interactive Single Page Applications (SPAs).
How we use it: Used to construct modular components like App.jsx, TabManager.jsx, ConceptGraphPanel.jsx, AnimatedRoadmap.jsx, and LogicFlowPanel.jsx. Manages state via useState, useEffect, and useRef without full-page reloads.

2. Node.js & npm (Runtime & Package Manager)

What it is: Node.js is a server-side JavaScript engine; npm is its package manager.
How we use it: Used during development to execute Vite development server (npm run dev), manage dependencies (axios, lucide-react, react-dom), and bundle production static assets.

3. FastAPI & Uvicorn (Backend Web Framework & ASGI Server)

What it is: FastAPI is a high-performance Python 3.11 web framework built on Starlette and Pydantic. Uvicorn is an Asynchronous Server Gateway Interface (ASGI) web server.
How we use it: Powers all backend HTTP REST endpoints (/auth, /workspaces, /documents, /graph, /content, /chat). Executes non-blocking async handlers and enforces strict input/output Pydantic schema validation.

4. PostgreSQL & SQLAlchemy 2.0 (Relational DB & ORM)

What it is: PostgreSQL is an enterprise object-relational database. SQLAlchemy is Python's premier Object-Relational Mapper (ORM).
How we use it: Persists relational data models (User, Workspace, Document, ExtractedText, ConceptNode, ConceptEdge, NodeMastery, ChatHistory). Uses JSONB columns for flexible non-relational node attributes and reference answer caching.

5. JSON (JavaScript Object Notation)

What it is: A lightweight data-interchange text format.
How we use it: Acts as the primary data exchange language across all system boundaries: HTTP REST requests/responses between React & FastAPI, structured prompt inputs/outputs for Gemini LLM, and JSONB columns in PostgreSQL.

6. Google Gemini API (gemini-2.5-flash LLM)

What it is: A state-of-the-art multimodal AI model provided by Google DeepMind.
How we use it: Performs multi-unit syllabus extraction, reference answer generation, MCQ quiz creation, AI Explain-Back active recall grading (0-100 score), and workspace-aware copilot chat answers.

7. JWT Authentication & bcrypt (Security Layer)

What it is: PyJWT provides JSON Web Token encoding/decoding; Passlib provides bcrypt password hashing.
How we use it: Hashes user passwords during signup, issues signed HS256 Bearer tokens upon login, and validates user authentication on every protected API route via FastAPI's Depends(get_current_user).

2. System Interconnections & Data Flow Architecture

The diagram below illustrates how raw user input flows from the React SPA frontend, through FastAPI middleware and controllers, to PostgreSQL and the Gemini API, and back as JSON data to render reactive UI updates:



```

+-----+-----+-----+
|                               | Axios HTTP Requests (JSON + JWT Authorization) |
|                               v                               |
+-----+-----+-----+
|                               FASTAPI ASGI BACKEND (Uvicorn) |
| Middleware: CORSMiddleware | Security: PyJWT Bearer Auth | Validation: Pydantic Schemas | | | |
| Routes:   auth.py | workspaces.py | documents.py | graph.py | content.py | chat.py |
| Services: graph_service.py | content_service.py | chat_service.py | gemini_service.py |
+-----+-----+-----+
|                               | SQLAlchemy ORM 2.0 | HTTP REST Calls (API Key) |
|                               v                               v                               |
+-----+-----+-----+
|                               POSTGRES SQL DATABASE | GOOGLE GEMINI API |
| Tables: users, workspaces, documents, | Model: gemini-2.5-flash |
| extracted_text, concept_nodes, concept_edges, | Generates: Unit Syllabus, JSON Graph, |
| node_mastery, generated_content, chat_history | Reference Answers, Explain-Back Grading |
+-----+-----+-----+

```

3. Deep-Dive Code Implementation & Annotations

A. Database Models — SQLAlchemy ORM (backend/app/models/concept_graph.py)

This file defines the relational tables for Concept Nodes, Prerequisites Edges, and Mastery States:

```

import uuid
from sqlalchemy import Column, String, Text, ForeignKey, Table, Integer, DateTime
from sqlalchemy.dialects.postgresql import UUID, JSONB
from sqlalchemy.orm import relationship
from app.database import Base

class ConceptNode(Base):
    __tablename__ = 'concept_nodes'

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    workspace_id = Column(UUID(as_uuid=True), ForeignKey('workspaces.id', ondelete='CASCADE'), nullable=False)
    title = Column(String(255), nullable=False)
    unit = Column(String(50), nullable=True, default='Unit 1')
    summary = Column(Text, nullable=True)
    sub_points = Column(JSONB, nullable=True) # JSON array of topic bullets
    answer_cache = Column(Text, nullable=True) # Caches AI generated markdown answer

    workspace = relationship('Workspace', back_populates='concept_nodes')
    mastery = relationship('NodeMastery', back_populates='node', uselist=False, cascade='all, delete-orphan')
    prerequisites = relationship('ConceptNode', secondary='concept_edges',
                                primaryjoin='ConceptNode.id==ConceptEdge.node_id',
                                secondaryjoin='ConceptNode.id==ConceptEdge.prerequisite_id')

```

B. DFS Cycle Resolution Algorithm (backend/app/services/graph_service.py)

Ensures AI-generated prerequisite links never form circular loops, enforcing a strict Directed Acyclic Graph (DAG):

```

def detect_and_resolve_cycles(nodes_dict, edges_list):
    """
    Runs Depth-First Search (DFS) back-edge detection.
    Strips out any back-edges pointing to nodes in current recursion stack.
    """
    adj = {n_id: [] for n_id in nodes_dict}
    for src, dst in edges_list:
        if src in adj:
            adj[src].append(dst)

    visited = set()
    rec_stack = set()
    clean_edges = []

    def dfs(u):
        visited.add(u)

```

```

rec_stack.add(u)
for v in adj.get(u, []):
    if v in rec_stack:
        # Back-edge detected! Skip this edge to break the cycle
        continue
    clean_edges.append((u, v))
    if v not in visited:
        dfs(v)
rec_stack.remove(u)

for node_id in nodes_dict:
    if node_id not in visited:
        dfs(node_id)
return clean_edges

```

C. Robust AI JSON Auto-Repair Parser (backend/app/services/gemini_service.py)

Handles unescaped quotes, Markdown fences, and truncated JSON responses from the Gemini API:

```

import json, re

def parse_json_from_ai(text: str) -> dict:
    """
    Multi-tiered JSON repair parser for AI responses.
    """
    # Step 1: Strip Markdown ```json ... ``` code fences
    cleaned = re.sub(r'```(?:json)?\s*(.*?)\s```', r'\1', text, flags=re.DOTALL).strip()

    # Step 2: Try direct json.loads with strict=False to allow raw control characters
    try:
        return json.loads(cleaned, strict=False)
    except json.JSONDecodeError:
        pass

    # Step 3: Extract outer JSON object / array using regex
    match = re.search(r'(\{.*\}|\[.*\])', cleaned, re.DOTALL)
    if match:
        try:
            return json.loads(match.group(1), strict=False)
        except json.JSONDecodeError:
            pass

    # Step 4: Fallback auto-closing brackets if truncated by LLM token limit
    open_braces = cleaned.count('{') - cleaned.count('}')
    open_brackets = cleaned.count('[') - cleaned.count(']')
    repaired = cleaned + (']' * open_brackets) + ('}' * open_braces)
    return json.loads(repaired, strict=False)

```

D. JWT Security & Password Hashing (backend/app/utils/security.py)

Handles secure bcrypt password hashing and HS256 JWT access token generation:

```

import jwt
from datetime import datetime, timedelta
from passlib.context import CryptContext
from app.config import settings

pwd_context = CryptContext(schemes=['bcrypt'], deprecated='auto')

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

def create_access_token(data: dict, expires_delta: timedelta = None) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=settings.JWT_EXPIRE_MINUTES))

```

```
to_encode.update({'exp': expire})
return jwt.encode(to_encode, settings.JWT_SECRET_KEY, algorithm='HS256')
```

E. React API Interceptor & Auth State (frontend/src/api.js)

Attaches JWT token to all HTTP requests and provides unified error handling:

```
import axios from 'axios';

const API_BASE = import.meta.env.VITE_API_URL || 'http://localhost:8000';
let authToken = localStorage.getItem('token');

export function setToken(token) {
  authToken = token;
  if (token) localStorage.setItem('token', token);
  else localStorage.removeItem('token');
}

async function request(endpoint, options = {}) {
  const headers = { 'Content-Type': 'application/json', ...options.headers };
  if (authToken) headers['Authorization'] = `Bearer ${authToken}`;
  const res = await fetch(`${API_BASE}${endpoint}`, { ...options, headers });
  if (!res.ok) {
    const err = await res.json().catch(() => ({}));
    throw new Error(err.detail || 'API Request Failed');
  }
  return res.json();
}
```

4. Technology Interconnection Matrix

Technology	Layer & Role	Connected Component	Input / Output Payload
React 18	Frontend SPA UI	Axios API Client (api.js)	User clicks $\rightarrow$ JSON REST payloads
FastAPI	Backend REST Server	React SPA & PostgreSQL	HTTP JSON Requests $\rightarrow$ Pydantic validation
SQLAlchemy	ORM Data Access	PostgreSQL Database	Python objects $\rightarrow$ SQL queries & JSONB
Gemini API	LLM Intelligence	gemini_service.py	Document context text $\rightarrow$ Structured JSON output
ReportLab	PDF Synthesis	graph.py route	PostgreSQL nodes $\rightarrow$ Binary PDF stream
PyJWT / bcrypt	Security Auth	auth.py & FastAPI Depends	Password string $\rightarrow$ HS256 Bearer Token